CSS Grid Implicit & Explicit Grid

Complete Guide: Easy English with Examples

Contents

Chapter 1

Introduction

CSS Grid has two ways of creating layouts: **Explicit Grid** and **Implicit Grid**. Understanding the difference is crucial for controlling your layouts!

Explicit Grid = You define the layout exactly. **Implicit Grid** = The grid automatically creates space for extra items.

Think of it like building a table: explicitly telling someone "make 3 columns and 2 rows," versus them automatically adding more rows when you add more data!

1.1 What's the Difference?

Feature	**Explicit Grid**	**Implicit Grid**
Defined by	You define it	Grid auto-creates
Control	Full control	Automatic
Content	Must fit grid	Adds new tracks
Properties	grid-template-*	grid-auto-*

1.2 Real-World Analogy

Explicit: "I want a 3-column bookshelf with 2 shelves."

Implicit: "I want 3-column width shelves. Add more shelves if I have more books."

Chapter 2

Understanding Explicit Grid

2.1 What is Explicit Grid?

The explicit grid is the space you **deliberately create** using:

- `grid-template-columns`
- `grid-template-rows`
- `grid-template-areas`

You tell CSS exactly how many columns and rows exist.

2.2 Defining Explicit Grid

2.2.1 Explicit Columns

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 150px;
  /* Creates exactly 3 columns */
}
```

2.2.2 Explicit Rows

```
.container {
  display: grid;
  grid-template-rows: 50px 100px;
  /* Creates exactly 2 rows */
}
```

2.2.3 Both Together

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 150px;
  grid-template-rows: 50px 100px;
  /* Grid with 3 columns and 2 rows */
}
```

2.2.4 Visual Result

```
 100px     200px      150px

 Item 1    Item 2     Item 3

 Item 4    Item 5     Item 6

   50px         100px (row heights)
```

2.3 Example 1: Fixed Explicit Grid

```
.dashboard {
  display: grid;
  grid-template-columns: 250px 1fr 250px;
  grid-template-rows: 60px 1fr 60px;
  gap: 15px;
  height: 100vh;
}

.header {
  grid-column: 1 / -1; /* Span all columns */
}

.sidebar {
  grid-column: 1; /* First column */
}

.main {
  grid-column: 2; /* Second column */
}

.footer {
  grid-column: 1 / -1; /* Span all columns */
}
```

2.4 Example 2: Explicit with Repeat

```
.grid {
  display: grid;
  grid-template-columns: repeat(4, 150px);
  grid-template-rows: repeat(3, 100px);
  /* Creates 4 columns x 3 rows */
  gap: 10px;
}
```

This is still explicit because you defined it!

Chapter 3

Understanding Implicit Grid

3.1 What is Implicit Grid?

The implicit grid is the **automatic space created** by the browser when:

- You have more items than explicit grid tracks
- Items are placed outside the defined grid
- The browser needs to create new rows/columns

3.2 When Implicit Grid Creates New Tracks

3.2.1 Example: More Items Than Columns

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 150px;
  /* Only 3 columns defined (explicit) */
  gap: 10px;
}

/* HTML with 6 items */
<div class="item">1</div>
<div class="item">2</div>
<div class="item">3</div>
<div class="item">4</div> <!-- New row! -->
<div class="item">5</div>
<div class="item">6</div>
```

3.2.2 Visual Result

```
Explicit Grid (3 columns):

 Item 1   Item 2    Item 3

  (row height auto-sized)
```

```
Implicit Grid (auto-created):

  Item 4    Item 5     Item 6

  (browser decides row height)
```

3.3 The Problem: Uncontrolled Sizing

Without controlling implicit tracks, sizes are unpredictable!

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 150px;
  /* No grid-template-rows defined */
  /* Item 4, 5, 6 go to implicit row */
  /* Row height = auto (fits content) */
}
```

Result: Implicit row might be different height than explicit rows!

Chapter 4

Controlling Implicit Grid

4.1 grid-auto-rows Property

Sets the size of automatically-created rows.

4.1.1 Syntax

```
grid-auto-rows: size;
```

4.1.2 Example 1: Fixed Row Heights

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 150px;
  grid-auto-rows: 100px;
  /* All implicit rows are 100px */
  gap: 10px;
}
```

All rows (explicit and implicit) now align!

4.1.3 Example 2: Flexible Row Heights

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 150px;
  grid-auto-rows: 1fr;
  /* Rows grow equally to fill space */
  gap: 10px;
  height: 500px;
}
```

4.1.4 Example 3: Minimum Row Height

```
.container {
  display: grid;
  grid-template-columns: 100px 200px 150px;
```

```
  grid-auto-rows: minmax(100px, auto);
  /* Rows at least 100px, grow for content */
  gap: 10px;
}
```

4.2 grid-auto-columns Property

Sets the size of automatically-created columns.

4.2.1 When Are Implicit Columns Created?

When you place an item outside the explicit column range!

4.2.2 Example

```
.container {
  display: grid;
  grid-template-columns: 100px 200px;
  /* Only 2 explicit columns */

  grid-auto-columns: 150px;
  /* If item placed in column 3+, use 150px */

  gap: 10px;
}

/* Item spans columns 1-4 */
.item-wide {
  grid-column: 1 / 5;
  /* Columns 3-4 are implicit */
}
```

4.2.3 Visual Result

```
Explicit:     Implicit:

 100   200    150      150

Col 1  Col2  Col 3    Col 4
```

4.3 Example: Grid with Both Auto Sizes

```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 200px);
  grid-template-rows: 250px;

  grid-auto-rows: 250px;
```

```
  /* All rows: 250px */

  grid-auto-columns: 200px;
  /* Extra columns: 200px */

  gap: 15px;
}

.item {
  background: lightblue;
  border-radius: 8px;
}
```

Result: Consistent grid regardless of item count!

Chapter 5

Explicit vs Implicit: Comparison

5.1 Side-by-Side Comparison

Aspect	**Explicit**	**Implicit**
How defined	You set it	Browser auto-creates
Properties	grid-template-*	grid-auto-*
Predictable	Yes	No (unless controlled)
Examples	Dashboards	Card galleries
Content	Fixed	Varies

5.2 When to Use Each

5.2.1 Use Explicit Grid When:

- Layout is fixed and known
- All items fit in the grid
- You need precise control
- Examples: Headers, footers, dashboards

5.2.2 Use Implicit Grid When:

- Content varies in quantity
- You want auto-wrapping
- Items flow naturally
- Examples: Image galleries, product lists

5.2.3 Use Both When:

- Main layout is explicit
- Content areas use implicit

- Example: Dashboard with card grid inside

Chapter 6

Real-World Examples

6.1 Example 1: Explicit Dashboard Layout

```
.dashboard {
  display: grid;
  grid-template-columns: 250px 1fr 250px;
  grid-template-rows: 60px 1fr 40px;
  gap: 15px;
  height: 100vh;
}

.header {
  grid-column: 1 / -1;
  background: #333;
  color: white;
  padding: 15px;
}

.sidebar {
  grid-row: 2 / 3;
  grid-column: 1 / 2;
  background: #f5f5f5;
  overflow-y: auto;
}

.main-content {
  grid-row: 2 / 3;
  grid-column: 2 / 3;
  padding: 20px;
}

.right-panel {
  grid-row: 2 / 3;
  grid-column: 3 / 4;
  background: #fafafa;
```

```
  border-left: 1px solid #ddd;
}

.footer {
  grid-column: 1 / -1;
  background: #f9f9f9;
  border-top: 1px solid #ddd;
  padding: 10px 15px;
  font-size: 12px;
  color: #666;
}
```

All positions are explicitly defined!

6.2 Example 2: Implicit Card Gallery

```
.card-gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  grid-auto-rows: 300px;
  /* Implicit: All rows are 300px */
  gap: 20px;
  padding: 20px;
}

.card {
  background: white;
  border-radius: 8px;
  overflow: hidden;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

.card-image {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

.card-content {
  padding: 15px;
  height: 100px;
  overflow: hidden;
}
```

Items automatically wrap, all rows same height!

6.3 Example 3: Hybrid Layout

```
.app {
  display: grid;
  /* Explicit header and footer */
  grid-template-rows: 60px 1fr 50px;

  /* Implicit content rows */
  grid-auto-rows: minmax(100px, auto);

  height: 100vh;
}

.header {
  grid-row: 1;
  background: #2c3e50;
  color: white;
}

.content {
  grid-row: 2;
  display: grid;
  /* Nested explicit grid for content */
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-auto-rows: 200px;
  gap: 15px;
  padding: 20px;
}

.footer {
  grid-row: 3;
  background: #f5f5f5;
  border-top: 1px solid #ddd;
}
```

Main layout is explicit, content uses implicit!

6.4 Example 4: Responsive with Both

```
.container {
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  grid-auto-rows: minmax(100px, auto);
  gap: 15px;
}

.header {
  grid-column: 1 / -1; /* Explicit: spans all */
```

```
  min-height: 60px;
}

.items {
  grid-column: 1 / -1;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  grid-auto-rows: 150px; /* Implicit: all same */
  gap: 15px;
}

.item {
  background: white;
  border-radius: 6px;
  padding: 15px;
}

.footer {
  grid-column: 1 / -1; /* Explicit: spans all */
  min-height: 50px;
}
```

Chapter 7

Grid Flow: Controlling Layout Direction

7.1 grid-auto-flow Property

Controls how items flow into implicit grid tracks.

7.1.1 Values

Value	Behavior
row (default)	Items flow left to right, then wrap down
column	Items flow top to bottom, then wrap right
dense	Fill gaps (changes visual order)

7.1.2 Example 1: Row Flow (Default)

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-auto-flow: row; /* Default */
  gap: 10px;
}

/* Items 1-3 fill first row, items 4-6 fill second row */
```

```
  1     2     3

  4     5     6
```

7.1.3 Example 2: Column Flow

```
.container {
```

```
  display: grid;
  grid-template-rows: repeat(3, 100px);
  grid-auto-flow: column; /* New! */
  gap: 10px;
}

/* Items 1-3 fill first column, items 4-6 fill second column */
```

```
  1    4

  2    5

  3    6
```

7.1.4 Example 3: Dense Packing

```
.container {
  display: grid;
  grid-template-columns: repeat(4, 100px);
  grid-auto-flow: dense; /* Fill gaps! */
  gap: 10px;
}

.item:nth-child(2) {
  grid-column: span 2;
}

/* Grid auto-rearranges to fill gaps */
```

```
Without dense:

  1      2     3    4   (4 is alone in row 2)

     (Item 4 moves down)

With dense:

  1      2     4    5   (4 moves up to fill gap)
```

Chapter 8

Common Scenarios

8.1 Scenario 1: Fixed Layout + Auto Content

```
/* Main layout: explicit */
.page {
  display: grid;
  grid-template-columns: 200px 1fr 200px;
  grid-template-rows: 60px 1fr 40px;
}

/* Content area: implicit cards */
.content {
  grid-column: 2;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  grid-auto-rows: minmax(200px, auto);
  gap: 20px;
}
```

8.2 Scenario 2: Responsive Grid

```
.responsive-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-auto-rows: minmax(200px, auto);
  gap: 15px;
}

/* Extra rows auto-created as needed */
/* All rows same height due to grid-auto-rows */
```

8.3 Scenario 3: Masonry-Like Layout

```
.masonry {
  display: grid;
```

```css
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  grid-auto-rows: 200px;
  grid-auto-flow: dense; /* Fill gaps */
  gap: 15px;
}

.item-tall {
  grid-row: span 2; /* Tall item */
}

.item-wide {
  grid-column: span 2; /* Wide item */
}

/* Dense packing fills gaps automatically */
```

Chapter 9

Debugging: Understanding Your Grid

9.1 Using Browser DevTools

9.1.1 Chrome/Edge DevTools

1. Open DevTools (F12)
2. Find your grid container in Elements
3. Look for grid icon next to class name
4. Click to overlay the grid
5. See explicit (solid) and implicit (dashed) tracks

9.1.2 Firefox DevTools

1. Open DevTools (F12)
2. Go to Inspector tab
3. Select grid container
4. Click grid badge
5. View explicit and implicit gridlines

9.2 Visual Inspection

```
/* Add background colors to see actual grid */
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
  gap: 10px;
  background: lightgray;
  padding: 10px;
}
```

```
.item {
  background: white;
  border: 2px solid blue;
}

/* Gaps and backgrounds show grid structure */
```

9.3 Common Grid Issues

9.3.1 Issue: Items Overflowing

Problem: Items go outside container
Solution: Check grid-template-columns defines enough space

9.3.2 Issue: Unexpected Row Heights

Problem: Implicit rows different size than explicit
Solution: Set grid-auto-rows to match

9.3.3 Issue: Wrong Item Placement

Problem: Items in unexpected positions
Solution: Check grid-auto-flow value

Chapter 10

Best Practices

10.1 Rule 1: Define Explicit Grid First

Always define the main layout explicitly:

```
.app {
  display: grid;
  grid-template-columns: 200px 1fr 250px;
  grid-template-rows: 60px 1fr 40px;
  /* Explicit: clear structure */
}
```

10.2 Rule 2: Control Implicit Tracks

Never leave implicit sizing to chance:

```
.gallery {
  grid-auto-rows: minmax(200px, auto);
  /* Controlled: minimum 200px */
}
```

10.3 Rule 3: Use grid-auto-flow for Direction

Make flow intention clear:

```
.vertical {
  grid-auto-flow: column; /* Obvious: flows down then right */
}
```

10.4 Rule 4: Combine Explicit + Implicit

Outer layout explicit, inner flexible:

```
.container {
  /* Explicit outer */
```

```
  grid-template-columns: 1fr;
  grid-template-rows: 60px 1fr 40px;
}

.content {
  /* Implicit inner */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  grid-auto-rows: 200px;
}
```

Chapter 11

Browser Support

Explicit and Implicit grids are supported in:

- Chrome 57+
- Firefox 52+
- Safari 10.1+
- Edge 16+
- Opera 44+

Note: Internet Explorer does NOT support CSS Grid.

Chapter 12

Quick Reference

12.1 Key Properties

Property	Purpose
grid-template-columns	Define explicit columns
grid-template-rows	Define explicit rows
grid-template-areas	Define named areas (explicit)
grid-auto-columns	Size implicit columns
grid-auto-rows	Size implicit rows
grid-auto-flow	Direction of implicit placement

12.2 Cheat Sheet

```
/* EXPLICIT: Define everything */
.explicit {
  grid-template-columns: 100px 200px 150px;
  grid-template-rows: 50px 100px;
}

/* IMPLICIT: Let browser create tracks */
.implicit {
  grid-auto-rows: 100px;
  grid-auto-columns: 200px;
}

/* EXPLICIT + IMPLICIT: Combine */
.hybrid {
  grid-template-columns: 200px 1fr;
  grid-auto-rows: minmax(100px, auto);
}
```

```css
/* EXPLICIT WITH AUTO ITEMS */
.content-grid {
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-auto-rows: 200px;
}

/* CONTROL FLOW */
.row-flow {
  grid-auto-flow: row; /* Default: left to right */
}

.column-flow {
  grid-auto-flow: column; /* Top to bottom */
}

.dense-flow {
  grid-auto-flow: dense; /* Fill gaps */
}
```

Chapter 13

Summary: Key Takeaways

- **Explicit Grid**: You define columns/rows with grid-template-*
- **Implicit Grid**: Browser auto-creates with grid-auto-*
- **Main Difference**: Control vs automation
- **Use Explicit**: For known, fixed layouts (dashboards)
- **Use Implicit**: For variable content (galleries)
- **Use Both**: Outer explicit, inner implicit
- **grid-auto-rows**: Control implicit row heights
- **grid-auto-columns**: Control implicit column widths
- **grid-auto-flow**: Control direction of placement
- **Always Control**: Never leave implicit sizing random

Chapter 14

Practical Workflow

14.1 Step 1: Plan Your Layout

Ask:

- Is the layout fixed or variable?
- How many main areas?
- Will items wrap?

14.2 Step 2: Define Explicit Grid

```
.container {
  display: grid;
  grid-template-columns: /* Main layout */
  grid-template-rows: /* Fixed heights */
}
```

14.3 Step 3: Control Implicit Tracks

```
.container {
  grid-auto-rows: minmax(100px, auto);
  /* If items overflow, rows are predictable */
}
```

14.4 Step 4: Set Auto Flow Direction

```
.container {
  grid-auto-flow: row; /* or column, or dense */
  /* How should items fill? */
}
```

14.5 Step 5: Test Responsive

Test with varying content amounts to ensure implicit grid behaves as expected!

Chapter 15

Conclusion

Understanding explicit and implicit grids is fundamental to mastering CSS Grid. The explicit grid gives you precision for main layouts. The implicit grid provides flexibility for content. Together, they create powerful, responsive designs.

Master both concepts and you'll never struggle with grid layouts again!